

Class 20: Model Deployment and Prediction Serving

Data-driven Systems Engineering

Agenda

- From a trained artifact to a callable, running service.
- Flask as a service layer, Streamlit as a presentation layer.
- The serving architecture already present in this repository.
- Serving patterns and production concerns beyond a single demo.
- Closing the Data-Driven block and opening the road into MLOps.

Learning goals

- Explain the roles of APIs and dashboards in exposing a trained model.
- Compare Flask and Streamlit and know when each is the right tool.
- List the minimum quality requirements for a credible prediction service.
- Recognize which production serving concerns this course's demos do not yet address.

Course Map and Running Cases



Today in the course

Focus: Turning a model artifact into a served, callable, and observable system.

Bridge: This is the last class of the Data-Driven block: everything from data type, through encoding, features, tuning, and evaluation, converges here into one deployable service boundary.

Fraud case

The fraud case becomes a Flask scoring endpoint, 'Class6_model_api.py', callable by other services or analysts.

Pasteurization case

The pasteurization case pairs a Flask streaming API with a Streamlit dashboard hub, 'streamlit/Main.py', so sensor state and predictions stay visible together.

Course thread: From Class 21 onward, the course treats these served systems as full software engineering objects: requirements, architecture, delivery, and monitoring.

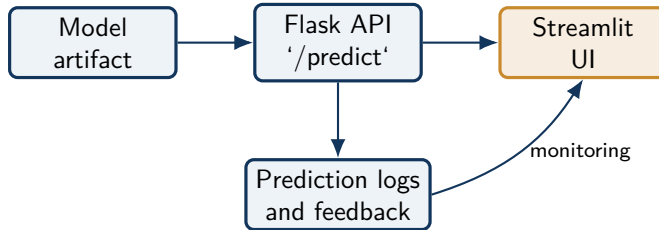
A model creates value only through an interface

- A trained model that nobody can call or interpret has limited practical value, however strong its offline metrics.
- Interfaces make predictions available to users, services, and operational workflows.
- The interface also shapes trust: input checks, response format, and clarity all matter.

Flask and Streamlit play different roles

- Flask** Best for programmable endpoints over HTTP, such as 'POST /predict' or a streaming route.
- Streamlit** Best for a fast interactive UI: demos, dashboards, or exploratory interfaces.
- Together** A common pattern is Flask for the backend service and Streamlit for the human-facing interface.

Repository serving architecture



Repository example: the fraud scoring endpoint

```
model = pickle.load(open("notebooks/best_random_forest_model.pkl", "rb"))

@app.route("/")
def index():
    return jsonify({"message": "Model API running"}), 200

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(force=True)
    features = np.array(data["features"]).reshape(1, -1)
    pred = model.predict(features)[0]
    return jsonify({"prediction": int(pred)})
```

- This is the complete serving logic of 'Class6_model_api.py': load once at startup, expose one prediction route.
- It is deliberately small so students can see exactly what a minimal service boundary contains, and what it is still missing.

What a better API should include

- A clear input schema, instead of trusting `'data["features"]'` to always be well-formed.
- Error handling for malformed or incomplete requests.
- Stable JSON output with prediction and confidence where appropriate.
- Logging hooks for later monitoring and debugging.
- Health or status endpoints for operational checks.

Repository example: the dashboard hub

```
st.set_page_config(
    page_title="440MI - Machine Learning Dashboard",
    page_icon=":robot:",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.title("Machine Learning Streamlit Hub")
st.markdown("""
Welcome to the 440MI + 305SM Course.
This interface automatically discovers all available modules
inside the pages/ directory and allows you to navigate between them.
""")
```

- 'streamlit/Main.py' is the single entry point that Streamlit's multipage mechanism uses to discover every dashboard in 'pages/'.
- One hub, many served views: fraud scoring, pasteurization streaming, and online learning all live behind it.

Serving patterns in production

Batch scoring	Predictions computed periodically over a stored dataset, not on demand.
Online request-response	A client calls an endpoint and waits synchronously for one prediction, as in 'Class6_model_api.py'.
Streaming serving	Predictions computed continuously over an incoming event stream, as in the pasteurization case.

Production serving concerns this repo's demos do not cover

- Latency budgets and load testing under concurrent requests.
- Model versioning and safe rollback if a new artifact regresses.
- Schema and input validation enforced before the model is ever called.
- Observability: structured logs, request tracing, and alerting on failure rates.

Why this matters now

These are exactly the gaps that motivate the software engineering and MLOps material starting next class.

From a demo service to a production deployment



- Every arrow here is a topic the MLOps block develops in depth, starting from Class 21.

Repo activity

- 'Class6_model_api.py' is the minimal Flask serving example for the fraud case.
- 'streamlit/Main.py' is the dashboard hub that presents fraud, pasteurization, and streaming pages together.
- 'pasteurization/serving.py' shows the same serving concerns applied to a continuous stream instead of a single request.

Readiness checklist before calling a model “served”

- Can the service reject a malformed request without crashing?
- Is the exact preprocessing used at inference identical to what was used in training?
- Is there any record of which model version answered a given request?
- Would a teammate other than the author know how to redeploy this service?

Question

'Class6_model_api.py' returns a bare 0/1 prediction with no schema, confidence, or logging. Is it production-ready?

- This is deliberately the same tension raised across evaluation (Class 17) and now serving: a working demo is not the same as a dependable system.

Papers and materials

Flask documentation; Streamlit documentation; The Twelve-Factor App; Sculley et al., Hidden Technical Debt in Machine Learning Systems; Practical MLOps.

From This Class to the Next

What stays with us

- A served model is a small software system: an interface, a preprocessing contract, and an operational surface, not just a '.pkl' file.
- Flask and Streamlit solve complementary problems: programmable endpoints versus human-facing presentation.
- The gaps in this course's demo services (validation, versioning, observability) are not oversights; they are exactly what the MLOps block exists to close.

Project use

Project teams should now be able to point at their own model artifact and describe, concretely, how it would be called, by whom, and what could go wrong.

Next connection

Class 21: Intervention Cybertec opens the MLOps block, moving from this course's own demo services into full software engineering practice for data products.